九阶 · Linux 与 Python 自学资料

11

AI 与 GPU 入门

从本地模型到 GPU 推理与监控

教科书

AYMLiu-Pzw · 2026-10-22

九阶 · Linux 与 Python 自学资料

第 11 册

AI 与 GPU 入门

从本地模型到 GPU 推理与监控

资料类型	教科书
适用读者	完成综合项目阶段 7 的学习者；GPU 条件可按正文替代
发布日期	2026-10-22
作者署名	AYMLiu-Pzw

阅读导航

按章级标题定位正文；PDF 阅读器中的书签同时提供章、节两级入口。

1. 8-01 GPU 是谁：训练 vs 推理、显存 vs 内存
2. 8-02 用 Ollama 起一个本地模型并对话
3. 8-03 在 Colab 借一张免费 GPU，看懂 nvidia-smi
4. 8-04 故意把显存撑爆：复现 CUDA out of memory（本册心脏）
5. 8-05 跑一个最小训练
6. 8-06 用 Python 调 DeepSeek API 做命令行助手
7. 8-07 用 vLLM 部署一个能访问的推理服务
8. 8-08 把 GPU 指标接进监控面板
9. 8-09 读 DeepSeek-V3 技术报告，写 MoE / MLA 中文笔记
10. 8-10 画一张全景图，讲 10 分钟（★）

11 • AI 与 GPU 入门

本册是阶段 8 的正文，从 8-01 起步；配套与路线见《17-关卡总表》。

证据等级	可提交什么	不能怎么写
实机证据	本机 / 免费 GPU 上实际跑出的命令、日志、截图或文件	不能用自己的推测替代实际输出
等价演练证据	在另一个可运行后端完成同一“服务化 / 监控 / OOM 方法”练习，并标注替代对象	不能伪称实测 CUDA 或 vLLM
只读理解证据	逐段解释配置、命令、指标关系，保留来源与自己的疑问	不能声称已部署或已读报告实证

每个 ★ 关会写明接受哪一级；**没有 GPU 不扣分，伪造验证据实不合规。**17 总表以本册同名节的证据要求为准。

本册共 10 关（8-01 ~ 8-10）。目标：会用 Ollama / vLLM 跑与部署模型，看得懂 nvidia-smi，能复现并排查 CUDA OOM，能把 GPU 指标接进监控，能画全景图讲 10 分钟。

注意：这一册必须在网上跑（Colab / Kaggle）：CUDA 相关命令（nvidia-smi、显存溢出复现）只认 NVIDIA 显卡——**如果你手上是 AMD 卡 / 没有独显，跑不了是正常的，不是你操作错了。**免费资源（Colab / Kaggle）覆盖到绝大多数关卡，不需要买显卡——**唯一例外是 8-08**：那一段要 Docker + NVIDIA 卡，免费档跑不了，书里给了「降级演练」的替代路径。

本册是什么：GPU 与 AI 运维 —— 分清训练与推理、看懂显存、把模型跑起来并接进监控。

动手做实验时照这两条来：实验闭环（先记录原状态 → 操作 → 按输出 / 退出码验证 → 恢复现场）和踩坑记录位（现象 → 原因 → 解决 → 一句话总结）——不必逐关都写，做完整册回头总结一次就够。

8-01 GPU 是谁：训练 vs 推理、显存 vs 内存

这一关不敲命令，只把三组词分清。它们是后面所有关卡的"地图"。

一、训练 vs 推理（最重要的区分）

阶段	在干什么	资源特点	你以后会怎么碰到
训练	拿海量数据反复调整模型参数	几天到几个月、几百张卡、**满负荷运转**	训练任务挂了/变慢了；要重启、要从断点续训
推理	拿训好的模型回答用户	长期在线、**显存吃紧**、要低延迟	推理服务起不来、变慢、显存不够、被限流

一句话记忆：**训练烧算力，推理吃显存**；训练是"批处理"，推理是"在线服务"。AI 运维的日常里，**推理占大头**（因为它在线上跑，随时随地会出问题）。

二、显存 vs 内存（两个不同的"内存"）

```
内存（RAM）  ← 主机/服务器的内存，给 CPU 用；用 ps / free 看
显存（VRAM） ← 显卡自己的内存，给 GPU 用；用 nvidia-smi 看
```

解读：模型参数、中间结果都放在**显存**里。显存不够，程序直接报错（8-04 的 OOM）。**主机内存再大也帮不上显存**——这是新手最容易混的一点。

三、为什么 GPU 比 CPU 快（并行结构与数据搬运）

CPU 像"几个博士"：单个算得很快，但人手少。GPU 像"几千个小学生"：单个算得慢，但**人多、能同时干同一件简单的事。**矩阵运算是"同样的简单计算重复几百万次"，正好是 GPU 的主场 —— 这就是 GPU 在 AI 里不可替代的原因。

完成判定

· 能用自己的话说清训练和推理的区别，并各举一个运维场景 · 能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇） · 能说出显存和内存是两回事，各用什么命令看 · 能用"博士 vs 小学生"讲清 GPU 为什么适合 AI（不掉进数学细节）

回滚方式：纯认知，无操作。

8-02 用 Ollama 起一个本地模型并对话

为什么先做这一关：它**零外部依赖**：不抢 GPU、不花钱、断网也能跑（模型下载时除外）。先在自己机器上把"模型跑起来"这件事做成，再谈故障。

步骤 1：装并拉一个小模型

```
$ sudo apt install -y zstd                          # 新装 Ubuntu 没带 zstd，官方脚本解包会报 "requires zstd"
$ curl -fsSLo /tmp/ollama-install.sh https://ollama.com/install.sh
$ less /tmp/ollama-install.sh                          # 先查看：确认来源可信、内容符合预期
$ sh /tmp/ollama-install.sh                            # 确认后再执行（需要管理员权限）
$ ollama --version
ollama version is 0.40.7       （2026-10-21 快照；以你机器实际输出为准）

$ ollama pull qwen2.5:1.5b        # 1.5B 的小模型，约 1GB，纯 CPU 也能跑
pulling manifest ...
success

$ ollama list
NAME             ID           SIZE    MODIFIED
qwen2.5:1.5b     a1b2c3d4e5f6  986 MB  10 seconds ago
```

解读：**1.5B = 15 亿参数**，属于"能塞进普通电脑"的体量。官方 Linux 安装会把服务作为系统用户 `ollama` 运行，模型默认存在 `/usr/share/ollama/.ollama/models`，之后断网也能用；可用 `OLLAMA_MODELS` 改目录。Docker 路线的模型则在容器的 `/root/.ollama`，由具名卷 `ollama` 持久化。

安装入口：先选一条路线

路线	何时选	最少执行
A 原生安装	能访问官方下载站，想用 systemd 管理	按上方三步：`curl ... -o /tmp/ollama-install.sh` → `less /tmp/ollama-install.sh` → `sh /tmp/ollama-install.sh` → `ollama pull qwen2.5:1.5b` → `ollama run qwen2.5:1.5b`
B Docker	官方脚本下载不稳，但 Docker 可用	`docker run -d --name ollama -v ollama:/root/.ollama -p 127.0.0.1:11434:11434 ollama/ollama` → `docker exec ollama ollama pull qwen2.5:1.5b` → `docker exec -it ollama ollama run qwen2.5:1.5b`
C 离线归档	目标机不能联网，但能拷入经校验的安装包	从 Releases 下载匹配平台/架构的归档，记录 tag 与校验值；Linux amd64 常见 `ollama-linux-amd64.tar.zst`，先从官方 Release 页面取得对应版本的校验值并本地比对，再用 `sudo tar --zstd -C /usr -xf 文件名` 解包

选好路线再回到下面的端口检查。A 路线若服务未启动，用 `sudo systemctl start ollama`；B 路线用 `docker start ollama`，后续 Ollama 命令加 `docker exec ollama ...`；C 路线的手工解包不会自动建立系统用户、unit 和卸载清单，长期使用仍以官方安装器或发行版包管理方案为准。三条路线都不通时，先跳过模型下载，用 08 册 5-02 的 Prometheus 靶子练“看端口 / 请求接口 / 停再起”；文档里注明本轮环境阻断，不把跳过写成模型已部署。

步骤 2：对话

```
# A / C 原生路线
$ ollama run qwen2.5:1.5b
# B Docker 路线
$ docker exec -it ollama ollama run qwen2.5:1.5b
>>> 用一句话解释什么是运维
运维就是保证线上的服务一直能正常跑、出问题能尽快修好的那批人。
>>> /bye
```

解读：跑起来就像个聊天工具。**但你要关注的不是聊天，是"这东西怎么对外提供服务"**——它默认在本地开了个 HTTP 接口（下一关/下一册就用得上）。

步骤 3：看它开了什么端口（运维视角）

```
$ ss -tuln | grep 11434
tcp   LISTEN 0 4096  127.0.0.1:11434  0.0.0.0:*

$ sudo ss -tulnp | grep 11434   # 需要看进程时再加 sudo
tcp   LISTEN 0 4096  127.0.0.1:11434  0.0.0.0:*  users:(("ollama",pid=3121,fd=3))

$ curl -s http://localhost:11434/api/generate -d '{"model":"qwen2.5:1.5b","prompt":"你好","stream":false}' |
head -c 300
{"model":"qwen2.5:1.5b","created_at":"2026-10-09T22:40:11Z","response":"你好！有什么可以帮你的吗？",...}
```

解读：三件事：① **11434 是 Ollama 的默认端口**（记住它，就像 9090 之于 Prometheus）；② 它只监听 `127.0.0.1`，所以只能本机访问（这是默认的安全设置）；③ 接口是 HTTP + JSON —— **跟 5-06 拉 Prometheus 指标是同一类活儿。**

步骤 4：制造一次"服务没起来"（提前熟悉故障）

```
# A 原生安装：用 systemd
$ sudo systemctl stop ollama
$ curl -s http://localhost:11434/api/tags
curl: (7) Failed to connect to localhost port 11434: Connection refused
$ sudo systemctl start ollama && curl -s -o /dev/null -w "%{http_code}\n"
http://localhost:11434/api/tags
200

# B Docker 安装：只操作容器
$ docker stop ollama
$ curl -s http://localhost:11434/api/tags
curl: (7) Failed to connect to localhost port 11434: Connection refused
$ docker start ollama && curl -s -o /dev/null -w "%{http_code}\n" http://localhost:11434/api/tags
200

# C 离线手工安装：由你自己在后台启动时，记下父进程 PID
$ /usr/bin/ollama serve > ~/lab/ollama.log 2>&1 &
$ OLLAMA_PID=$!
$ kill -TERM "$OLLAMA_PID"   # 验证 Connection refused 后再启动
$ /usr/bin/ollama serve >> ~/lab/ollama.log 2>&1 &
$ OLLAMA_PID=$!
```

解读：**Connection refused = 端口没人听 = 服务没起来**（对比 4-09 的端口排查链）。排错也按路线：A 看 `journalctl -u ollama`；B 看 `docker logs ollama`；C 看你自己记录的 `~/lab/ollama.log`。Docker 路线没有宿主 systemd unit，不能执行 `systemctl stop ollama`。

完成判定

· 模型拉下来并能对话（截图或记录一句回答） · 能说出模型文件放在哪、默认端口是多少、只听哪个地址 · 能用 curl 调一次接口拿到 JSON · 能故意停掉服务、看到 Connection refused、再恢复

回滚方式：A 原生路线执行 `ollama rm qwen2.5:1.5b` 与 `sudo systemctl stop ollama`；B Docker 路线执行 `docker exec ollama ollama rm qwen2.5:1.5b`、`docker stop ollama`，如需连具名卷一起清理再执行 `docker rm ollama && docker volume rm ollama`；C 离线路线用实际安装的 `ollama rm`，并用你启动时记录的 PID 停止进程。

8-03 在 Colab 借一张免费 GPU，看懂 nvidia-smi

这一关的目标：不用买显卡，也能在真 GPU 上敲第一条命令，并且**看懂 nvidia-smi 的每一列**——那是 GPU 运维的"仪表盘"。

步骤 1：开一张免费 GPU

```
浏览器打开 https://colab.research.google.com → 新建笔记本
菜单：代码执行程序 → 更改运行时类型 → 硬件加速器选 **GPU** → 保存
（国内网络优先用 Kaggle（能直连）：https://www.kaggle.com → New Notebook → Settings → Accelerator 选 GPU，同样免费；Colab 需要能访问 Google 的网络，连不上是网络问题、不是你操作错）

先打印平台实际给你的卡型和运行环境，再记录，不要求必须是 T4：
!nvidia-smi --query-gpu=name,driver_version,memory.total --format=csv
!python -c "import torch; print(torch.__version__, torch.version.cuda, torch.cuda.get_device_name(0))"

在第一个单元格里输入（Colab 里命令前面加 !）：
!nvidia-smi
```

抢不到卡不是你操作失误：免费 GPU 有额度，高峰时段可能分不到 —— 遇到 "You have exhausted your GPU quota" 或给不到 GPU，换时段（或换 Kaggle）再试即可。

步骤 2：把 nvidia-smi 的每一列读出来

```
+-----------------------------------------------------------------------------------------+
（下面这份输出来自 2025 年 T4 示例；免费平台不保证卡型/驱动/CUDA 版本，读法不变，务必记录自己本次实际输出）
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   42C    P0              26W /   70W |     412MiB /  15360MiB |      0%      Default |
+-----------------------------------------+------------------------+----------------------+
```

看什么	含义 / 什么时候要盯它
GPU 型号（示例 Tesla T4）	不同型号显存和算力差很多；报工单时要写清实际型号
Driver / CUDA Version	Driver Version 是驱动版本；标题里的 CUDA Version 是驱动**最高支持的 CUDA 版本**，通常不等于系统实际安装的 CUDA Toolkit。还要分别记录 Runtime/Toolkit 与 PyTorch 自身的 CUDA 构建版
Temp（42C）	温度；长期高温（>80C）要查散热和负载
W / Cap（26W/70W）	功耗与上限；"卡在跑但功耗很低"往往说明它在等数据（见 8-04 延伸）
Memory-Usage（412MiB/15360MiB）	**显存用了多少 / 总共多少**——这一栏是 AI 运维最常盯的数字
GPU-Util（0%）	算力利用率：**高=在算，低=在等**（等数据、等通信、等磁盘）

步骤 3：看看"有人在用"时是什么样

```
!pip install -q torch
import torch, time
print("GPU 可用：", torch.cuda.is_available(), torch.cuda.get_device_name(0))
x = torch.randn(8000, 8000, device="cuda")   # 造一个大矩阵扔进显存
for _ in range(30):
    y = x @ x                                  # 反复做矩阵乘
!nvidia-smi          # 另开一个单元格：看 GPU-Util 和显存的变化
```

解读：跑起来时你会看到 **GPU-Util 飙到 90%+、显存占用涨上去**；跑完又掉下来。**学会"看曲线和数字判断它在干活还是在等"**，就是 GPU 运维的基本功。

步骤 4：记录一次环境信息（这是产出）

```
# 把这一组抄进你的笔记（以后报工单/排查都要写；值以本次运行输出为准）
1. GPU 型号与实际显存：
2. 驱动版本：
3. nvidia-smi 显示的 CUDA 上限：
4. 实际 CUDA Runtime / Toolkit（如有）：
5. PyTorch 版本 / PyTorch 的 CUDA 构建版 / torch.cuda.is_available()：
```

完成判定

· 能在 Colab（或 Kaggle）里拿到任一可用 GPU 并跑出 `nvidia-smi` · 能逐列说出含义（至少：型号、驱动/CUDA、温度、功耗、显存、利用率） · 能区分驱动支持上限、实际 CUDA Runtime/Toolkit 与 PyTorch CUDA 构建版 · 能记录一次完整环境信息与实际卡型 · 知道免费卡型会变、抢不到卡时该怎么办

回滚方式：Colab 会话关掉即回收；不留本机痕迹。

8-04 故意把显存撑爆：复现 CUDA out of memory（本册心脏）

为什么它是心脏：**OOM（显存溢出）是 AI 环境最常见的资源故障之一。**本关要亲手制造一次（或做等价演练），读出报错中的证据，再提出至少四种缓解办法；重点是诊断顺序，不是背定义。

环境兜底（抢不到免费卡也能完成本关）：本关优先在真 GPU 上复现 CUDA OOM；**配额用尽、平台排队时别干等**——

① **同方法降级演练**：用 `ulimit -v` 先把试炼进程的内存上限压小、再申请超大内存，得到干净的 `MemoryError`：`bash -c "ulimit -v 500000; python3 -c 'x = bytearray(20 * 1024**3)'"`；想经历“被系统 OOM 杀”就用限内存容器（有 Docker 时）：`docker run --memory=256m python:3.14 python3 -c 'x = bytearray(20 * 1024**3)'`（退出码 137 = OOMKilled）。这是**内存/容器 OOM 降级演练，不是 CUDA OOM**；可借用“现象 → 线索 → 建议”的读法，但不能把报告、退出码或内存单位混写成 CUDA 数据。**WSL 用户特别注意：别在本机直接“申请超大内存”赌系统杀进程**；要么用 `ulimit -v` 限住，要么把演示关进限内存容器。

② 记录里如实标注：“**未实测 CUDA，用内存 OOM 做了同方法演练**”；

③ 完成判定分两档：真 GPU 运行得到 CUDA OOM，或用降级路径得到内存 OOM 并明确标注。不能把后者写成前者。

步骤 1：把显存吃满

```
import torch
MiB = 1024 ** 2
free_before, total = torch.cuda.mem_get_info()
print("运行前：", round(free_before / 1024**3, 2), "GiB 空闲 /",
      round(total / 1024**3, 2), "GiB 总量")

# 先合法占住当前空闲显存的约 75%，再从剩余空间之外请求一块更大的张量
hold = torch.empty(int(free_before * 0.75), dtype=torch.uint8, device="cuda")
free_now, total = torch.cuda.mem_get_info()
request = free_now + 256 * MiB
print("请求：", round(request / 1024**3, 2), "GiB：当前空闲：",
      round(free_now / 1024**3, 2), "GiB")
x = torch.empty(request, dtype=torch.uint8, device="cuda")

RuntimeError: CUDA out of memory. Tried to allocate 3.74 GiB.
GPU 0 has a total capacity of 14.56 GiB of which 3.49 GiB is free.
Of the allocated memory 10.46 GiB is allocated by PyTorch,
and 112.00 MiB is reserved by PyTorch but unallocated.
If reserved but unallocated memory is large try setting
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation.
```

（上面的数字只是同一次 T4 运行的示例。你运行时必须保存**同一次输出**里的四组数：Tried to allocate、total/free、allocated、reserved；不同卡型与占用状态不可混拼。代码先合法占住一部分显存，再请求“当前空闲 + 256 MiB”，所以不依赖“大小恰好 20000 或 62000”，在不同显存的卡上都应越过当前空闲边界。）

逐句读这段报错（这就是"看懂报错"）：

原句	它在说什么
`Tried to allocate 3.74 GiB`	**现象**：这一次想申请 3.74 GiB
`total capacity of 14.56 GiB of which 3.49 GiB is free`	**线索**：总容量 14.56 GiB，当前只剩 3.49 GiB；本次请求比剩余空间多 256 MiB，因此必然失败
`10.46 GiB is allocated by PyTorch`	**线索**：前面的 hold 已经合法占用了约 10.46 GiB；这不是“刚启动就 0 占用却失败”的矛盾状态
`try setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`	**建议**：官方给的缓解办法之一（治碎片化）

读报错三步法（读一行报错）：先看**现象**（要了多少、在哪失败）→ 再看**线索**（还剩多少、谁占的）→ 最后看**建议**（官方解法常常就写在最后几行）。14 册《日志报错解读》讲的就是这个顺序。

步骤 2：四种缓解办法，一个一个试

```
# 办法 1：把尺寸改小（最直接）
size = 12000
x = torch.randn(size, size, device="cuda")
print("占用：", round(torch.cuda.memory_allocated() / 1024**3, 2), "GiB")

# 办法 2：用完就释放（避免"垃圾"占着显存）
del x
torch.cuda.empty_cache()

# 办法 3：数据分批（batch 变小）—— 训练里最常用
# 原来一次喂 64 条 → 改成 16 条，显存占用大致降到 1/4

# 办法 4：治碎片化（报错里建议的那个环境变量）
# 注意：它只在"启动前"生效—写成启动命令的一部分最稳：
#   PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True python train.py
# 已经跑起来的会话里再改下面这行 os.environ，不会重新生效（要重启进程）
import os
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"
```

解读：四种办法对应四种病因 —— **要得太多（尺寸）/ 没及时释放（泄漏）/ 一次喂太多（batch）/ 内存碎片（碎片化）**。遇到 OOM 时先按这四类整理证据，再决定调整尺寸、生命周期、batch 或碎片策略，而不是只想到换卡。

步骤 3：复现"明明还有显存却还是 OOM"

```
a = torch.randn(4000, 4000, device="cuda")   # 4000² × 4B = 64 MB ≈ 0.06 GiB
b = torch.randn(4000, 4000, device="cuda")
del a
b = b * 1.0                                  # 制造一些碎片
try:
    c = torch.randn(3000, 3000, device="cuda")
    print("这次成功")
except RuntimeError as e:
    print("失败：", str(e).splitlines()[0])
torch.cuda.empty_cache()                     # ← 关键动作
print("清理后：", round(torch.cuda.memory_reserved() / 1024**3, 2), "GiB 仍被保留")
```

解读：这就是报错里那句 "reserved but unallocated" 的真实含义 —— **显存被 PyTorch 抓着但没用上。**看到这个关键词，第一反应就是 `empty_cache()` 或调 batch。

（这段是"演示"不是"必修"：这套 64MB 级的张量在 15GB 的 T4 上跑，**大概率打印"这次成功"**——碎片型 OOM 要接近满载时才会出现，小尺寸看到成功属正常；关键是理解 `reserved` 与 `allocated` 的差别。）

步骤 4：把这次排查写成记录（产出）

```
# 显存溢出排查记录
现象：报 CUDA out of memory，Tried to allocate X GiB
线索：报错里"还剩多少空闲"最关键 —— 剩得多说明要得太狠或碎片化；剩得少说明真不够
四类病因与对策：
  1）尺寸/模型太大     → 减小输入、换更小模型、量化（参数用 8-bit 等低精度存，省显存）
  2）没及时释放        → del + torch.cuda.empty_cache()
  3）一次喂太多        → batch_size 调小、梯度累积
  4）内存碎片          → PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
验证：改完后跑通，并打印 memory_allocated() 确认占用在预期范围
一句话总结：OOM 先读"还剩多少"，再按四类病因对号入座
```

完成判定

· **证据二选一**：实机 CUDA OOM 原文，或内存 OOM 原文 + “未实测 CUDA，使用等价演练”标注 · 先预测批次变化，再完成报错的现象 / 线索 / 建议拆读 · 能说出四种缓解办法及各自对应的病因 · 写出这份排查记录 · 能说清 `reserved but unallocated`（实机时）或内存 OOM 与显存 OOM 的差异（等价路径时）

回滚方式：Colab/Kaggle 里"运行时 → 重新启动会话"即释放全部显存；本机跑就杀掉进程。

8-04 预测与排错：合上教程，先写一句预测“批量大小翻倍时，显存和首个失败点怎样变”；再从自己的报错记录里删掉“处理”段，只看现象和线索，从四种病因中选两项各给一个验证动作。完成后逐条对照；没有 GPU 时同样可以对内存 OOM 做，但报告标题必须写“等价演练”。

延伸：训练变慢了怎么拆（生产里最典型的"模糊故障"）

为什么单独讲它：前面几关的故障都是"坏了"（起不来、连不上）。而真实工作里更常见的是"没坏，但慢了"——没有报错、没有红灯，只有一句"今天训练怎么这么慢"。**这类故障考的就是"把模糊问题拆成可验证的假设"**，正是实际工作里最需要的能力。

分层拆法（从上到下，一层一层看）：

层	先看什么	慢了通常说明什么
① **GPU 计算**	`nvidia-smi` 的 GPU-Util 是不是长期接近 100%	长期 100% = 计算是瓶颈（正常情况）；**很低** = GPU 在等数据/等通信，问题在下面几层
② **显存**	显存占用是否已接近上限、有没有 swap / OOM 前兆	显存吃满会让框架反复腾挪、变慢（8-04 的 OOM 是它的极端形态）
③ **通信（多卡）**	NCCL 日志里各卡等待时间是否拉长	某张卡拖后腿（慢卡/坏卡）会让所有卡一起等——木桶效应
④ **网卡 / 网络**	多机训练时网卡流量、是否有丢包/重传	参数同步要走网络；网络抖动直接表现为"训练一步忽快忽慢"
⑤ **存储 / 读数据**	数据盘 IO、checkpoint 目录的写入速度	checkpoint 写满带宽时，训练会明显卡顿（这就是为什么常看到"每存一次 checkpoint 就慢一下"）
⑥ **数据加载**	DataLoader 的 worker 数、CPU 是否打满	读数据跟不上 GPU：GPU 利用率低 + CPU 高，最典型的"饿着显卡"

怎么用它（假设 → 验证）：

```
现象：训练比昨天慢了一倍
假设 1：GPU 在等数据   → 验证：nvidia-smi 里 GPU-Util 只有 20%、CPU 打满 → 成立：加大 DataLoader worker
假设 2：某张卡拖后腿   → 验证：nvidia-smi 里 8 张卡里 1 张显存/利用率异常 → 成立：报修/换卡，先把它排除
假设 3：checkpoint 拖慢 → 验证：把保存间隔调长再跑一段，速度回升 → 成立：把 checkpoint 放到更快的盘 / 错峰
```

一句话记住：**先分清是"GPU 在干活"还是"GPU 在等"** —— 在干活（利用率高）说明瓶颈可能在计算本身；在等，就往"数据 / 通信 / 存储"三层去找。

顺带一提 • 断点续训（checkpoint / 断点续训）
是什么：训练过程中定期把"模型参数 + 优化器状态 + 走到第几步"存成文件（checkpoint）；万一机器挂了或者要停机维护，从最近一次 checkpoint 接着练，不用从头来。
为什么大模型必须要它：一次训练要跑几天到几个月、占用成百上千张卡 —— 中途不出任何问题几乎不可能。**没有它，一次断电就等于烧掉几百万。**所以"能不能从断点恢复"是大模型训练的生死线。
真要用时去哪查：搜"PyTorch checkpoint resume training"或"deepspeed save_checkpoint"；顺手记一句：**存 checkpoint 会抢磁盘带宽，所以经常能看到"每存一次就慢一下"**（这正是上面那张分层表第 ⑤ 层）。
顺带一提 • NCCL 通信延迟抖动 与 checkpoint 耗时
NCCL 是什么：多张卡之间"对答案"用的通信库（集合通信）。各卡算完一批梯度要互相汇总，谁慢了大家都得等——所以叫"通信延迟抖动"。
为什么要盯它：训练忽快忽慢、多卡利用率参差不齐，很多时候根因就在通信上（某张卡慢、某条链路抖），而不是模型的问题。**生产里的日常，盯的就是它们。**
checkpoint 耗时：写一次 checkpoint 会同时吃掉存储带宽，短则几秒、长则几十秒，期间训练几乎停顿 —— 高频保存会明显拉低整体吞吐。
真要用时去哪查：搜"NCCL debug log"（设 `NCCL_DEBUG=INFO` 看每步耗时）和"checkpoint 加速"（异步保存 / 更快的数据盘）；先知道有这两个东西、知道它们会拖慢训练，就够这一轮用了。

8-05 跑一个最小训练

为什么只跑两轮：你不需要学会训练模型 —— **只需要"跑过一次真的训练"，这样 8-04 的显存问题才有具体场景**，也能对照观察训练与推理的负载差异。所以就用最小的 MNIST，两轮收工。

步骤 1：两轮训练跑起来

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

tf = transforms.ToTensor()
train = datasets.MNIST(root="./data", train=True, download=True, transform=tf)
loader = DataLoader(train, batch_size=64, shuffle=True)

model = nn.Sequential(nn.Flatten(), nn.Linear(784, 128), nn.ReLU(), nn.Linear(128, 10)).cuda()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(2):                          # 就两轮
    for i, (x, y) in enumerate(loader):
        x, y = x.cuda(), y.cuda()
        loss = loss_fn(model(x), y)
        opt.zero_grad(); loss.backward(); opt.step()
        if i % 200 == 0:
            print(f"epoch {epoch} step {i} loss {loss.item():.3f}")

epoch 0 step 0 loss 2.315
epoch 0 step 200 loss 0.421
...
epoch 1 step 800 loss 0.187
```

解读：**loss 在往下走 = 训练在正常进行**（2.3 → 0.19）。这一步只要跑得通、看得懂 loss 下降，就达标了。

步骤 2：和显存对上号（把两关连起来）

```
print("batch_size=64 时显存占用：", round(torch.cuda.max_memory_allocated()/1024**2, 1), "MiB")

torch.cuda.reset_peak_memory_stats()          # ← 清掉上一次的峰值，否则第二次读到的还是历史最高

# 把 batch_size 改成 256 再跑一次，看占用变成多少：
loader2 = DataLoader(train, batch_size=256, shuffle=True)
（重复上面的训练循环，只跑 50 步）
print("batch_size=256 时显存占用：", round(torch.cuda.max_memory_allocated()/1024**2, 1), "MiB")
```

（这个 MLP 很小，峰值只有**几十 MiB**——用 GiB 两位小数会显示成 0.0，所以这里改用 MiB；两次之间要 `reset_peak_memory_stats()` 才看得出变化。）

解读：**batch 越大、显存占用越大**（大致成正比）。这就是 8-04 里"办法 3：数据分批"的现实依据 —— 显存不够时，把 batch 调小是训练侧最常用的手法。

完成判定

· 两轮训练跑通，loss 明显下降 · 能说出 loss 下降说明什么 · 用两个不同 batch_size 各跑一次，说出显存占用的变化趋势 · 能说出"batch 大小 → 显存 → OOM"这条因果链

回滚方式：Colab 会话重启即清；下载的 MNIST 数据在 `./data`，删掉即可。

8-06 用 Python 调 DeepSeek API 做命令行助手

这一关的产出是一个可复用的命令行小工具：命令行里敲一行，模型回答并记进日志。它把阶段 3 的 requests / argparse / logging 和阶段 8 的"模型服务"接在一起。

步骤 1：拿一个 API Key（没有也能做完）

```
1) 注册 https://platform.deepseek.com → API Keys → 新建，复制（只显示一次）
2) 不要写进代码！放进环境变量：
   $ echo 'export DEEPSEEK_API_KEY="sk-你的key"' >> ~/.bashrc && source ~/.bashrc
   $ echo 'export DEEPSEEK_MODEL="deepseek-flash"' >> ~/.bashrc && source ~/.bashrc
3) 验证 key 只打印前缀，不要把完整值贴到公开日志：
   $ echo $DEEPSEEK_API_KEY | head -c 6
4) 模型名不写死在代码里；默认用 deepseek-flash，执行前再看官方模型页并设置 DEEPSEEK_MODEL
5) 要充值：按量计费；新账号不一定有免费额度。401 通常是 key 无效，402/余额不足是账户余额问题，两者不要混报
6) 本教程的模型名核对日期：2026-10-21（官方模型与价格页当时列出 deepseek-flash / deepseek-v4-pro）

# 不想充钱也行：用 8-02 的本地 Ollama 当"备胎"（下面脚本加 --local，两种后端都支持）
```

安全红线：Key 是钱，**绝不提交进 GitHub**。提交前用 `git status` 看一眼，必要时在 `.gitignore` 里加 `.env`、`.env.*`、`*.env` 三行（和 16 册的写法一致）。

步骤 2：脚本（两种后端都支持，可直接复制）

```
#!/usr/bin/env python3
"""命令行小助手：--ask "问题"；默认走 DeepSeek，--local 走本地 Ollama"""
import argparse
import json
import logging
import os
import sys

import requests

DEEPSEEK = "https://api.deepseek.com/chat/completions"
OLLAMA = "http://localhost:11434/api/generate"
DEEPSEEK_MODEL = os.environ.get("DEEPSEEK_MODEL", "deepseek-flash")
```

同一脚本续：模型调用函数

```
def ask_deepseek(prompt, timeout=30):
    key = os.environ.get("DEEPSEEK_API_KEY")
    if not key:
        raise RuntimeError("没有找到 DEEPSEEK_API_KEY（见步骤 1）")
    resp = requests.post(
        DEEPSEEK,
        headers={"Authorization": f"Bearer {key}", "Content-Type": "application/json"},
        json={"model": DEEPSEEK_MODEL,
              "messages": [{"role": "user", "content": prompt}]},
        timeout=timeout,
    )
    resp.raise_for_status()
    data = resp.json()
    return data["choices"][0]["message"]["content"]


def ask_local(prompt, timeout=120):
    resp = requests.post(OLLAMA, json={"model": "qwen2.5:1.5b",
                                       "prompt": prompt, "stream": False}, timeout=timeout)
    resp.raise_for_status()
    return resp.json()["response"]
```

同一脚本续：主流程

```
def main():
    p = argparse.ArgumentParser(description="命令行小助手")
    p.add_argument("--ask", required=True, help="要问的问题")
    p.add_argument("--local", action="store_true", help="改用本地 Ollama")
    p.add_argument("--timeout", type=int, default=30)
    args = p.parse_args()

    logging.basicConfig(filename=os.path.expanduser("~/assistant.log"), level=logging.INFO,
                        format="%(asctime)s %(levelname)s %(message)s")
    try:
        answer = ask_local(args.ask, args.timeout) if args.local else ask_deepseek(args.ask, args.timeout)
    except Exception as exc:
        logging.error("调用失败：%s", exc)
        print(f"[错误] {exc}", file=sys.stderr)
        return 1
    logging.info("问：%s | 答：%s", args.ask, answer[:80])
    print(answer)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

解读（三处关键）：① **key 从环境变量读**，不写进代码；② **两种后端用一个开关切换**——没 key 时用 `--local` 照样能跑通，这就是 8-02 那一关的价值；③ 异常与日志走 3-09 的套路，退出码 0/1。

步骤 3：跑三种情况

```
$ python3 assistant.py --ask "用一句话解释什么是显存"
显存是显卡自带的内存，专门存放模型和数据，不够就会报 CUDA out of memory。

$ tail -1 ~/assistant.log
... INFO 问：用一句话解释什么是显存 | 答：显存是显卡自带的内存…

$ python3 assistant.py --local --ask "你好"
你好！有什么可以帮你的吗？

$ DEEPSEEK_API_KEY="" python3 assistant.py --ask "测试"
[错误] 没有找到 DEEPSEEK_API_KEY（见步骤 1）          ← 缺少 key 路径

# 再从下面的路径中至少复测一条；输出以官方当前错误格式为准
$ DEEPSEEK_MODEL="不存在的模型" python3 assistant.py --ask "测试"    # 模型不存在：HTTP 4xx
$ python3 assistant.py --timeout 1 --ask "测试"                      # 网络过慢：requests 超时异常
```

实测量化：成功路径、缺少/无效 key、余额不足、超时是四类不同结论。本轮执行环境没有 DeepSeek API Key，**没有做真实付费请求**；教材只把模型名更新为 2026-10-21 官方页面的值，并用无 key/本地后端静态验证了错误处理路径。读者有 key 时按上面命令各跑一次，记录实际 HTTP 状态码、错误正文、耗时与模型名；不要把 401、402 或超时都写成“key 不对”。

完成判定

· 脚本能带 `--ask` 与当前 `DEEPSEEK_MODEL` 提问并打印回答（或明确记录本轮未实测） · `--local` 能切到本地模型（没 key 也能演示） · 能区分缺少 key、认证失败、余额不足和超时 · 出错时给中文提示、日志有记录 · key 与模型名都没有写死在代码里 · 能说出"为什么 key 要放环境变量"

回滚方式：删掉脚本与日志即可；key 在环境变量里，随时可在控制台吊销。

8-07 用 vLLM 部署一个能访问的推理服务

为什么要 vLLM：Ollama 适合"自己玩玩"，而**生产上跑推理服务常用 vLLM**——它专为高并发推理优化。这一关让你从"跑模型"升级到"**部署一个服务**"：有端口、有接口、有日志、能排障。

环境提示：vLLM 需要 Linux + NVIDIA 显卡 + CUDA。**证据二选一**：① 在 Colab / Kaggle 免费 GPU 上实际启动 vLLM，提交 `/v1/models` 与一次请求的实机记录；② 用 Ollama 做等价“服务化”演练，提交端口监听、`/api/tags` 200、一次生成请求和日志，并写清“未实测 CUDA / vLLM”。两种都接受，但判定与提交物按路径分别执行，不能拿 Ollama 记录冒充 vLLM 实测。

步骤 1：起服务（Colab 或本机 GPU 机器）

```
$ pip install -q "vllm==0.32.0"       # 2026-10-18 发布快照；先确认 CUDA/PyTorch 兼容
$ vllm serve Qwen/Qwen2.5-1.5B-Instruct \
    --host 127.0.0.1 \
    --port 8000 --gpu-memory-utilization 0.8 > vllm.log 2>&1 &
$ tail -n 20 vllm.log
INFO: Started server process [1234]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000
```

解读：这里用当前的 `vllm serve` 子命令；`python -m vllm.entrypoints.openai.api_server` 入口已被上游标记为弃用。`--model` 选模型、`--host 127.0.0.1` 只监听本机、`--port` 选端口、`--gpu-memory-utilization 0.8` 把目标显存占用限制在约 80%，给其他进程留余量。启动输出已重定向到 `vllm.log`，后面 `tail -f` 才有文件可看。

安全边界：默认只绑定 `127.0.0.1`，不要直接改成 `0.0.0.0` 暴露到公网。远程访问应走私网/VPN、反向代理与防火墙白名单，并明确鉴权策略；`--api-key` 不能代替网络边界，也不能假设它保护了所有管理/辅助路径。

步骤 2：验证它能用

```
$ curl -s http://localhost:8000/v1/models | head -c 200
{"object":"list","data":[{"id":"Qwen/Qwen2.5-1.5B-Instruct","object":"model",...}]}

$ curl -s http://localhost:8000/v1/chat/completions -H "Content-Type: application/json" -d '{
  "model": "Qwen/Qwen2.5-1.5B-Instruct",
  "messages": [{"role": "user", "content": "用一句话说明什么是推理服务"}]
}' | head -c 300
{"id":"chatcmpl-...","choices":[{"message":{"role":"assistant","content":"推理服务是把训练好的模型部署成接口，让用户或程序随时调用。"}}]}
```

解读：**vLLM 提供的接口和 OpenAI 的格式一样**（`/v1/chat/completions`）—— 这意味着你 8-06 的脚本只要把 URL 换一下就能指向自己的服务。这是行业惯例，实战里也常见。

步骤 3：故意让它起不来，然后排障（三种原因）

```
# ① 端口被占（最常见）
$ vllm serve Qwen/Qwen2.5-1.5B-Instruct --host 127.0.0.1 --port 8000
OSError: [Errno 98] Address already in use
→ 排查：ss -tuln | grep 8000；看进程再加 sudo ss -tulnp | grep 8000
→ 处理：换端口（--port 8001）或停掉占用者

# ② 显存不够
ValueError: The model's max seq len (32768) is larger than the maximum number of tokens
that can be stored in KV cache. Try decreasing max_model_len.
（文案随 vLLM 版本变化：新版写成 "… Try increasing gpu_memory_utilization … or decreasing max_model_len …"——认 KV cache / max_model_len / gpu_memory_utilization 这几个关键词就够）
→ 排查：nvidia-smi 看还剩多少显存      → 处理：加 --max-model-len 4096 或调小 --gpu-memory-utilization

# ③ 模型路径/名字写错
OSError: Can't load the configuration of 'Qwen/Qwen2.5-1.5B'
→ 排查：确认模型名拼写、有没有下载权限      → 处理：改回正确名字
```

解读：**"端口 / 显存 / 模型路径"就是推理服务起不来的三大类原因**——和 4-09 的容器排障是同一套思路，只是换了对象。

步骤 4：把它当服务来管（这才是运维的部分）

```
$ ss -tuln | grep 8000              # 先确认 8000 是否有监听；进程细节再用 sudo ss -tulnp
$ tail -f vllm.log                 # 盯日志（别只看启动那几行，要看请求进来时的输出）
$ nvidia-smi                       # 另开一个终端盯显存和利用率
```

解读：**起服务只是第一步，能"看着它跑、出问题能查"才是运维。**这也是行业里常说的“黑盒变透明”。

完成判定

· **实机路径**：`vllm serve` 启动成功，`/v1/models` 返回模型列表，`vllm.log` 有记录，curl 拿到回答。
· **等价路径**：Ollama 端口在听，`/api/tags` 返回 200，一次生成请求成功，记录 Ollama 日志与对 vLLM 参数的对应理解，并标注“未实测 CUDA”。
· 两条路径都要完成 8-07 迁移检查：写出启动→监听→查模型→发请求→查日志顺序，并先预测停服务后的现象。
· 都要说清 127.0.0.1 的安全边界、模型服务起不来的三类原因及排查命令、OpenAI 兼容接口的意义；实机再解释 `--gpu-memory-utilization`。

回滚方式：只停本终端启动的后台作业 `kill %1`；若编号已丢失，先核对 PID 和命令归属再处理。Colab 会话关掉即回收。

8-07 迁移检查：选择一条证据路径后，合上书写出“启动 → 监听 → 查模型 → 发请求 → 查日志”的验证顺序；故意停一次服务，先预测现象，再恢复。等价路径还要指出 Ollama `/api/tags` 与 vLLM `/v1/models` 都是“列出可用模型”，但证据不能互相冒充。

8-08 把 GPU 指标接进监控面板

把两条线接起来：8 册你搭了 Prometheus + Grafana，但那时面板上只有 CPU / 内存 / 磁盘。**这一关把 GPU 也接上去**——"黑盒变透明"，这也正是行业里的常见说法。

步骤 1：GPU 指标要有个“翻译官”

```
node-exporter 管 CPU/内存/磁盘，但它**看不到 GPU**。
GPU 指标要用 dcgm-exporter（NVIDIA 官方）或 nvidia_gpu_exporter。

# 需要一台能跑 docker --gpus 的 NVIDIA 机器（本机 N 卡 / 按需租的云 GPU）；
# Colab、Kaggle 免费档没有 Docker，走下面的 gpu-mock 降级路径。
$ nvidia-smi                                # 第一步：驱动本身正常
$ docker run --rm --gpus all ubuntu:24.04 nvidia-smi   # 第二步：NVIDIA Container Toolkit 已接通 Docker
$ cd ~/lab/monitor
$ nano docker-compose.yml   # 在 services: 下增加 dcgm-exporter；不要放到 volumes: 后面
# 粘贴到 services: 内：
  dcgm-exporter:
    image: nvcr.io/nvidia/k8s/dcgm-
exporter:4.8.4@sha256:148b0c025e5f2850256816fa33754fbf4733b7a086697a95613fc0ba3db5b003
    container_name: dcgm-exporter
    ports:
      - "127.0.0.1:9400:9400"
    cap_add:
      - SYS_ADMIN
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    restart: unless-stopped
$ docker compose up -d dcgm-exporter prometheus
$ docker compose ps dcgm-exporter    # 应为 Up
$ curl -s http://localhost:9400/metrics | grep -m3 DCGM_FI_DEV
DCGM_FI_DEV_GPU_UTIL{gpu="0",...} 0
DCGM_FI_DEV_FB_USED{gpu="0",...} 412
DCGM_FI_DEV_FB_FREE{gpu="0",...} 14948
```

NVIDIA Container Toolkit 前置：若第二步报 could not select device driver ... with capabilities: [[gpu]]，先按 NVIDIA 官方容器工具安装页配置 apt 源，再执行 sudo apt install -y nvidia-container-toolkit、sudo nvidia-ctk runtime configure --runtime=docker、sudo systemctl restart docker，然后重跑第二步。不要用“直接装 CUDA Toolkit”代替这一步。官方页：https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html。

镜像与权限：这里固定 dcgm-exporter 4.8.4 的 tag 与 2026-10-21 核对到的 manifest digest，避免 latest 漂移。cap_add: [SYS_ADMIN] 仅用于 NVIDIA 官方镜像的专用实验环境；宿主发布端口只绑 127.0.0.1，不要裸暴露公网。Prometheus 与 exporter 加入同一个 Compose 项目后直接按服务名抓取，**不经过宿主回环地址**。

环境兜底（没有卡也能过关）：选择**实机 DCGM**或**gpu-mock 文件指标**之一。两者都使用同一个 Prometheus 容器和同一张 Grafana 图，只切换 target 与 metrics_path；降级证据必须标注“环境限制 / 未实测 GPU”。

gpu-mock 的可执行步骤：

① 新建目录和模板：

```
$ mkdir -p ~/lab/monitor/gpu-metrics
$ cat > ~/lab/monitor/gpu-metrics/gpu.txt <<'EOF'
# HELP DCGM_FI_DEV_GPU_UTIL Simulated GPU utilization percent.
# TYPE DCGM_FI_DEV_GPU_UTIL gauge
DCGM_FI_DEV_GPU_UTIL{gpu="0"} 5
# HELP DCGM_FI_DEV_FB_USED Simulated used framebuffer MiB.
# TYPE DCGM_FI_DEV_FB_USED gauge
DCGM_FI_DEV_FB_USED{gpu="0"} 512
# HELP DCGM_FI_DEV_FB_FREE Simulated free framebuffer MiB.
# TYPE DCGM_FI_DEV_FB_FREE gauge
DCGM_FI_DEV_FB_FREE{gpu="0"} 14848
EOF
# ② 在 docker-compose.yml 的 services: 内加入静态文件端点：
  gpu-mock:
    image: python:3.14-slim
    container_name: gpu-mock
    command: ["python3", "-m", "http.server", "9400", "--bind", "0.0.0.0", "--directory", "/metrics"]
    volumes:
      - ./gpu-metrics:/metrics:ro
    ports:
      - "127.0.0.1:9400:9400"
    restart: unless-stopped
$ docker compose up -d gpu-mock prometheus
$ curl -s http://localhost:9400/gpu.txt
$ docker compose exec prometheus wget -qO- http://gpu-mock:9400/gpu.txt | head -3
```

降级 job：在 prometheus.yml 的 scrape_configs: 下放一条即可；实机则把 target 改成 dcgm-exporter:9400 并删掉 metrics_path。

```
  - job_name: "gpu"
    metrics_path: /gpu.txt
    static_configs:
      - targets: ["gpu-mock:9400"]
```

让两块面板产生可验收的曲线（在宿主机运行约 36 秒；值明确标注为模拟）：

```
$ python3 -u - <<'PY'
from pathlib import Path
import time

path = Path.home() / "lab/monitor/gpu-metrics/gpu.txt"
samples = [(5, 512, 14848), (20, 2048, 13312), (85, 8192, 7168), (40, 4096, 11264)]
for util, used, free in samples:
    path.write_text(
        "# TYPE DCGM_FI_DEV_GPU_UTIL gauge\n"
        f'DCGM_FI_DEV_GPU_UTIL{{gpu="0"}} {util}\n'
        "# TYPE DCGM_FI_DEV_FB_USED gauge\n"
        f'DCGM_FI_DEV_FB_USED{{gpu="0"}} {used}\n'
        "# TYPE DCGM_FI_DEV_FB_FREE gauge\n"
        f'DCGM_FI_DEV_FB_FREE{{gpu="0"}} {free}\n',
        encoding="utf-8",
    )
    print(f"模拟指标已更新：util={util}, used={used}, free={free}")
    time.sleep(12)
PY
```

解读：DCGM_FI_DEV_GPU_UTIL = 利用率，FB_USED / FB_FREE = 显存已用 / 剩余（单位 MiB）。这些数字现在变成了 Prometheus 可抓取的指标；文件端点必须使用 .txt，并以同项目服务名访问。

步骤 2：让 Prometheus 抓它

```
$ cd ~/lab/monitor
$ nano prometheus.yml
# 在已有 scrape_configs: 下追加；实机 target 为 dcgm-exporter:9400，
# gpu-mock 降级时才使用上面的 metrics_path: /gpu.txt 与 gpu-mock:9400。
  - job_name: "gpu"
    static_configs:
      - targets: ["dcgm-exporter:9400"]
$ docker compose up -d --force-recreate prometheus
$ docker compose exec prometheus wget -qO- http://dcgm-exporter:9400/metrics | head -3
$ curl -s http://localhost:9090/api/v1/targets | grep -o '"health":"up"' | head -1
→ 打开 http://localhost:9090 → Status → Targets，gpu job 应为 UP
```

解读：Prometheus 和 exporter 在同一个 Compose 网络内，`dcgm-exporter:9400` 与 `gpu-mock:9400` 都由 Docker DNS 解析；宿主 `127.0.0.1:9400` 只供本机 curl 验证，不参与容器抓取。普通 `up -d` 可能复用旧容器，本关用 `--force-recreate prometheus`；若启用 `--web.enable-lifecycle`，也可用 `curl -X POST http://localhost:9090/-/reload` 后复查 Targets。

步骤 3：在 Grafana 上画出显存曲线

```
Grafana → New panel → 查询框输入：

  100 * DCGM_FI_DEV_FB_USED / (DCGM_FI_DEV_FB_USED + DCGM_FI_DEV_FB_FREE)

标题改成"GPU 显存使用率" → Save
# 再配一条：DCGM_FI_DEV_GPU_UTIL → 标题"GPU 利用率"
```

解读：这条 PromQL 和 5-04 学的磁盘使用率**是同一种句型**（用了多少 ÷ 总量 × 100）—— 只是把 df 的数字换成了显存的数字。

步骤 4：造一次负载，看曲线动

```
$ python3 -c "
import torch, time
x = torch.randn(6000, 6000, device='cuda')
t0 = time.time()
while time.time() - t0 < 60:
    x = x @ x
"
# 回 Grafana：GPU 利用率飙高、显存曲线有一块台阶 — 截图存档
```

完成判定

· 能说出 node-exporter 看不到 GPU、该用 DCGM exporter（实机）或文件指标端点（等价演练）
· **实机路径**：驱动与 Container Toolkit 验证通过，Prometheus 直接抓同网 `dcgm-exporter:9400` 且 GPU job 为 UP，Grafana 有显存/利用率面板和负载曲线。
· **等价路径**：按模板生成 `gpu-metrics/gpu.txt`，启动 gpu-mock 并让 Prometheus 抓 `gpu-mock:9400/gpu.txt`；GPU job 同样为 UP，两块面板出现模拟曲线，页面与文件名都标注“环境限制 / 未实测 GPU”。
· 完成 8-08 预测与迁移：解释 Targets UP 不等于有负载，并在改 job 名后说清 Targets 与查询的变化。
· 两条路径都能说清 GPU PromQL 与 5-04 磁盘查询共享“指标名 + 标签 + 计算”的句型。
回滚方式：实机执行 `docker compose rm -sf dcgm-exporter`，降级执行 `docker compose rm -sf gpu-mock`，再删除对应的 compose 服务段与 prometheus.yml 的 gpu job；执行 `docker compose up -d --force-recreate prometheus`，并复查 Targets 与查询已恢复。
8-08 预测与迁移：先回答“为什么 Targets UP 不等于 GPU 一定在负载运行”；再选择一条证据路径，把 job 名改成 `gpu-practice`，预测 Targets 与 Grafana 查询要改哪一处。执行后保留 UP 记录；若走 `gpu.txt`，逐行指出“指标名 / 标签 / 值”各是什么。

8-09 读 DeepSeek-V3 技术报告，写 MoE / MLA 中文笔记

参考：*DeepSeek-V3 Technical Report*，arXiv:2412.19437。本关要求**至少阅读与 MoE、MLA 直接相关的章节与图表**；下方大白话只建立入口，不算原文证据。8-06 的助手可用于解释术语、生成问题清单和核对理解，不能作为事实来源替代报告。
这一关不是让你啃论文，而是回答"聊聊你对 AGI 的理解"。你只需要**用自己的话讲清两个词**，再补一点自己的疑问。

步骤 1：先建立"大模型很贵"的直觉

```
模型越大 → 参数越多 → 显存越吃紧、算得越慢、花钱越多
于是大家想两件事：
  ① 怎么让**参数多但每次只用一部分**？ → 这就是 MoE
  ② 怎么让**长文本不要太占显存**？     → 这就是 MLA
```

解读：先有这个直觉，再看术语就不会晕 —— **MoE 和 MLA 都是"省"的招数**，一个省算力，一个省显存。

步骤 2：两个词的大白话版

词	大白话	为什么 AI 运维要懂
MoE（混合专家）	模型里有很多"专家小组"，来一个问题**只叫相关的几个专家**干活，其他不动 —— 参数量可以做到很大，但每次只跑一小部分	它决定了**显存占用和显存分布**：专家要分散在不同卡上，专家之间的调度和通信就是运维要盯的东西
MLA（多头潜在注意力）	把"注意力"要记的东西**压缩后再存**，同样长的上下文，占的显存小很多	它直接影响**长文本能不能跑得起来**：同样的卡，能不能开更长的上下文

遇到不懂的词（KV cache、注意力、token…）**别去啃整篇论文** —— 用 8-06 的那个命令行助手问一句"用大白话解释 KV cache 是什么"，然后记下来。

步骤 3：写一页笔记（产出）

```
# DeepSeek-V3 技术报告读书笔记
1) 它想解决什么：参数规模大 → 训练和推理都贵
2) MoE 用它干什么：参数多、但每次只激活一部分（省钱、省算力）
3) MLA 用它干什么：压缩注意力的存储（省显存、能开更长上下文）
4) 和运维的关系：专家怎么分布到多卡、显存怎么切、通信怎么不拖后腿
5) 我自己的疑问（至少一条）：比如"专家分布不均会不会导致某张卡特别忙？"
— 把疑问留在笔记里，并写清回到原文哪一节继续核对
```

完成判定

· 能用大白话讲清 MoE 和 MLA 各解决什么问题 · 能说出它们和“显存 / 卡 / 通信”的关系 · 笔记中 MoE、MLA 各至少有一条**原文证据**：章节号或章节名 + 图表编号 + 一句“它证明了我的哪个理解” · 至少一条自己的疑问，并写清去看原文哪里才能回答 · 命令行助手记录只作为术语解释，不作为报告事实来源

回滚方式：纯阅读与笔记，无风险。

读书笔记的最小证据格式：观点：… | 原文位置：第 X 节 / Figure Y / Table Z | 原文证据说明了什么：… | 我还想问：….。请给 MoE、MLA 各写一条。报告版本或章节标题不同就用你实际打开版本的编号与标题，不凭记忆补。

8-10 画一张全景图，讲 10 分钟（★）

为什么这一关是 ★：前面各关分别看 GPU、显存、网络、存储和调度；这一关要求把它们连成“从计算到调度”的六层地图，并能说明每层看什么指标、常见故障从哪里查。

步骤 1：照这六层画（一层一行，别抄术语，用你自己的话）

```
① GPU 计算       — 卡在不在干活？（nvidia-smi: GPU-Util；工具：8-03）
② 显存           — 装得下吗？（FB_USED/FB_FREE；故障：OOM，8-04）
③ NVLink / 卡间  — 多卡之间怎么快速换数据（同一台机器内的"高速通道"）
④ 网卡 / 网络    — 多机之间怎么同步（NCCL 走网络；抖动会让所有卡一起等）
⑤ 存储           — 数据和 checkpoint 从哪来、写多快（写 checkpoint 会抢带宽）
⑥ 调度           — 谁来分配这些卡和资源（K8s 的 requests/limits，6-07）
```

画法建议：一张 A4 横放，左边画这条"从算力到调度"的竖线，右边每一层挂上：**① 看什么指标 ② 出问题什么样 ③ 我练过哪一关。**手画比打字好（讲的时候指着说）。

步骤 2：每层准备一个"这里会出什么问题"

```
① GPU 计算：利用率常年个位数 → 在等数据（查 DataLoader / CPU）
② 显存：    OOM → 尺寸/batch/碎片/泄漏四类病因（8-04 的记录直接搬过来）
③ NVLink：  某张卡慢 → 所有卡跟着等（木桶效应）
④ 网络：    训练一步忽快忽慢 → 通信延迟抖动（NCCL 日志）
⑤ 存储：    每存一次 checkpoint 就卡一下 → 存储带宽被抢
⑥ 调度：    Pod 一直 Pending → requests 要得比节点能给的多（6-07 见过）
```

步骤 3：录音讲一遍（10 分钟，对着图不看稿）

```
讲法骨架（照这个顺序说，10 分钟足够）：
1) 一句话总起：AI 运维盯的是一条从算力到调度的链子（30 秒）
2) 从上往下走六层，每层讲"看什么指标 + 出问题什么样 + 我练过什么"（6 分钟）
3) 举一个真实例子：显存 OOM 我复现过，是这样查的（2 分钟）
4) 我的疑问 / 想深入的方向（1 分钟）
— 录音存进 learning-journey，方便以后对比进步
```

完成判定

· 有一张手画的全景图（六层齐全） · 每层能说出一个"会出什么问题"和一个"看什么指标" · 能对着图不卡壳讲满 10 分钟（录音为证） · 至少能回溯到三个自己练过的关卡（点名编号）

回滚方式：无操作。

阶段 8 • 学习总结练习：一张全景图 + 一次显存故障的完整交代

业务场景：组长（或未来的同事）问你："你说你懂 GPU，那我给你一张卡，你怎么知道它好不好？出了显存问题你怎么查？" —— 你要在 **10 分钟**里把这两个问题讲完，并且手里有实物。

阶段 8 单一验收卡

能力目标：把模型服务、GPU 资源、监控证据串起来，并诚实区分实机与等价演练。

必交证据：① Ollama 服务请求记录；② 8-04 的 OOM 记录（实机 CUDA，或内存 OOM + 等价标注）；③ 8-07 的 vLLM 实机记录，或 Ollama 等价服务证据；④ 8-08 的 Prometheus GPU job UP + 两块 Grafana 面板，可直接用 DCGM，也可用标注清楚的 `gpu.txt` 降级路径；⑤ 8-09 的 MoE / MLA 各至少一条原文证据（章节号或章节名 + 图表编号 + 理解说明，证明了自己的哪个理解；助手解释不作为原文事实）；⑥ 六层全景图；⑦ 10 分钟讲述稿/录音。

无 GPU 边界：可以完成等价演练，但不得声称见过真卡或实测 CUDA；如实际能进 Colab/Kaggle GPU，则补 `nvidia-smi` 截图作为实机证据。

不纳入本阶段：真实生产集群权限、付费多卡、Alertmanager 通知投递。

综合要求：① 用 Ollama 起一个小模型并调通；② 运行 8-04 并留下可追溯的 OOM 记录；③ 完成 8-07 的一条证据路径；④ 完成 8-08 的一条证据路径；⑤ 完成 8-09 的 MoE / MLA 原文证据与理解说明；⑥ 画六层全景图；⑦ 录 10 分钟讲述。

验收：· Ollama 有请求记录 · 8-04 记录标明真实 CUDA 或内存等价演练 · 8-07 按实机 / 等价路径提交对应输出并标真口径 · 8-08 的 GPU job 为 UP，两块面板存在，实机与降级证据均按本册同名节标注 · 8-09 的 MoE / MLA 各至少一条原文证据，均写清章节号或章节名、图表编号和理解说明；助手解释没有被当作原文事实 · 六层全景图 + 10 分钟录音 · 能说出“训练变慢了先看哪一层”

产出：仓库 `ai-inference-notes`（OOM 排查记录 + MoE / MLA 原文证据笔记 + 全景图 + 讲述稿 + 截图）

知识迁移：这张“从算力到调度”的地图用于理解 AI 服务的资源链路；显存问题只是其中一个高频切面，排查时仍要回到端口、日志和资源证据。

证据怎样命名和归档

关卡	建议文件	必须能看出的信息
8-02	`8-02-ollama.txt`	模型名、POST 请求、返回 JSON；Docker 路线注明容器命令
8-04	`8-04-oom.txt`	真实 CUDA 或内存等价演练、原始报错、四类病因
8-07	`8-07-serving.md`	监听端口、模型接口、一次请求与日志，注明实机或等价路径
8-08	`8-08-target.png`	GPU job 为 UP、target 是服务名、两块 Grafana 面板

文件名带关卡号，记录里写清命令、时间、退出码和环境边界；截图只作辅助，关键证据仍保留可读文本。

本册复习卡

① 模型服务同时看端口、请求日志与资源；显存不足是容量、批次、碎片、泄漏四类问题之一，不是单一开关。
② 服务没回答 → 先查端口监听，再查启动日志；GPU 监控 DOWN → 先查 Targets，再查 exporter 是否拿到了指标。
③ 独立变式：任选 8-04 与 8-08 各写一条“证据二选一”的提交清单，并指出哪一种写法属于伪造实机证据；正文不给清单答案。